

Task Checklist for Almost Any Machine Learning Project

A cheat sheet of tasks and things to take care of for every end-to-end ML projects

Written by Selva Prabhakaran | 8 min read

Share:



A cheat sheet of tasks and things to take care of for every end-to-end ML projects.

In this, I write down a check list of items and tasks to check whenever you start with a new Data Science / ML project.

Once you start off with the project there will be so many things going on. It is easy to lose sight on things that matter.

So, use this checklist periodically in your review meetings. This will help you stay organized, reduce missed steps and mistakes and ensure a better output consistently.

Usually, if you are a senior level manager / Data Scientist, you will typically hear about the business pain point. You need to convert the business problem to a data science problem.

Examples:

(i). Too many customers unsubscribing / annoyed → predict click through probability and select target group.

(ii). Excess inventory in warehouse → Forecast demand accurately, predict optimal safety stock level.

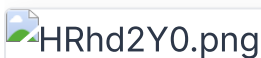
Then **Define the data science problem.**

- 1. Is machine learning the right approach** to solve this? How about a quick rule based solution?
- 2. What is the problem type:** Supervised / Unsupervised / Clustering (segmentation) / Recommendation / Optimization. It could be a combination of this as well.
- 3. Decide the label** (the response variable, Y) you want to predict.
- 4. What are the primary solutions** that will be developed?
- 5. Do you foresee following roadblocks?** (data availability on time, lack of infra to handle data, SME support to answer your business related questions, involvement of necessary stakeholders to ensure adoption)
- 6. What metrics to measure performance?**
- 7. What is the success criteria?**
- 8. Is there any assumptions** not spelled out?

Determine what data you will need for ML model. Based on this, you need to find where to source the required datasets from.

- 1. Identify the data sources:** Internal and external.

2. **How much data** is ideal to have vs bare minimum required?
3. Can it be loaded in your local computer? or need a **big data** / cloud environment?
4. Do you have the **permissions to use the data**?
5. Can **sourcing be completely automated**? or a manual data pull needed?
6. What is the **type of data**? Tabular / Image / Text etc
 - If tabular, is it cross sectional data or longitudinal?
7. What is the **primary key**? That is, at what level is each row unique?
8. What **format to store the data and where**?
9. **How frequently** new data arrives? check for duplicates, overwriting past history.
10. When multiple datasets / sources are involved, **map out the associations / architecture** and common keys like below.



Note: Above image is based on Brazillian e-Commerce dataset from Kaggle.

For the project to be deployed in production, the data sourcing and processing pipelines should be completely automated.

1. Identify what **data transformations** may be useful?
 - Discretize continuous variables, esp if there is a inherent meaning to the category.
 - Numeric encoding of categorical variables: One-hot encoding, Target encoding
 - Box Cox transformation / Yeo-Johnson Transform

2. Feature Scaling

- Min-max scaling / Standard scaling / Mean centering

3. Drop low/zero variance features

4. Drop id features like customer id, name etc unless they contain value info like gender from the salutation etc.

5. Impute missing data. Can create a new feature indicating whether the value was missing.

6. Handle outliers if needed

7. Extract features from **dates** and **geo-location**

Dates Features: Day of week, holiday, Weekday, week_no, days until holiday, holiday week.

Geo-location features: Distance between two points (places of interest) – Euclidean, Geodesic, Manhattan distance, zipcode of the geolocation, city, state, Country, Geographical clusters, Geohash, location based embeddings, number of neighbours (ex: airbnb properties nearby within radius). Ref:

[Geographic Data Science](#)

(https://geographicdata.science/book/notebooks/12_feature_engineering.html), [plainenglish](#)

(<https://python.plainenglish.io/how-to-use-geo-location-for-feature-engineering-using-near-by-points-of-interest-563752e6ad26>).

The main motives of this phase is to:

1. Understand and **summarize the data**
2. Understand **relationships** between features
3. Draw business/problem focused **insights**.

You should get a sense of the predictive power of the data and draw insights that puts things in perspective.

Check following points

1. Identify the data types and **fix formatting**. Ex: parse the **dates**.
2. **Optimize datatype** if needed, ex: Age can be stored as int8 instead of int64.
3. **Summarize the data** for each feature: Min, max, median, quartiles, # missing, # zeros, standard deviation etc, as necessary.
4. **Visualize relationships** (<https://machinelearningplus.com/plots/top-50-matplotlib-visualizations-the-master-plots-python/>) such as correlations, deviation, ranking, distribution, composition, change, groups.
5. **Calculate correlations**. Perform statistical tests where needed.
6. **Identify customer segments / cohorts** if any and draw insights associating it with the target variable and other features.

1. For classification problems: Check for **class imbalance**. Check naive prediction's performance. Decide imbalance handling strategies.

2. Decide appropriate **evaluation metrics to monitor**.

- **Classification:** F1 score, precision and recall, AUROC, Deciles capture rate, KS-statistic, log-loss etc
- **Regression:** MSE, MAPE, correlation of actual vs predicted etc.

1. **Build Base line model**. Consider Regularized logistic regression for classification, Random Forest for regression.
2. Use **k-fold cross validation** to while checking performance.
3. Consider **regularization methods** to counter overfitting.
4. Capture feature importances – **Does the most important feature make business sense?**. If not try out another ML algorithm and see what it says. If importances align with business understanding, keep the new model, else, recheck the data quality. Check the predictive power of features.
5. Avoid **data leakage**. Suspect if model performance is too good.

6. Capture performance of various **experiments**.
7. **Do you need only point estimates** of predictions or probabilities of getting specific value? How about confidence intervals and prediction intervals?

1. Identify data points that your model **predicted incorrectly**.

What information is needed for your model to predict them correctly?

Find the commonalities amongst incorrect predictions and come up with features that can help model to predict those data points correctly. Discuss the initial results and elicit ideas from your business counterparts / clients. Let them know you are in the process of improving your models performance.

2. Consider creating **innovative features**

Example ideas for new features: lifecycle of customers, adstock variables, lag variables for time series problems, location based, **date** based, interaction variables, squares, square roots, logs of important features, invent new (meaningful) formulas involving two or more key features, consider target encoding for categorical features.

3. Identify meaningful **cohorts** and build ML models for each instead of building for the entire dataset.
4. Try **Ensemble models**. XGBoost, GBM, Catboost, Light GBM,
5. **Tune hyperparameters** for performance on validation sample.
6. Test top models on unseen data (**hold out**).
7. Test models with data for **extreme situations** – for different times of the year, holiday peaks, value of predictors that has not happened yet but could.

1. Print **feature importances (FI)**. Variables that matter should ideally have high FI.

2. **Partial Dependence plots (PDP)**
3. **Individual Conditional Expectation (ICE) Plots**
4. Individual feature's contribution to a given prediction with **SHAP** values
5. Local Interpretable Model Agnostic Explanations (**LIME**)
6. **Tree Interpreter**
7. Accumulated Local Effects (**ALE**) Plots
8. **Capture rates** for deciles (for binary classification)
9. **Confusion matrices**
10. Build dashboard **simulating how your Y will change** when various X changes.

1. **Save and organize your model files** and configurations for each model refresh.
2. Model deployment may be of following types:
 - a. Serve model output as **REST API** (Flask, Fast API)
 - b. **Batch scoring** on a dataset and store results on a database.
 - c. **Integrate with mobile apps**, IoT etc.
3. Connect and integrate with data sources and setup a **data pipeline, CI/CD pipeline** (Circle CI, Jenkins, Travis CI etc), if needed.
4. **Manage dependencies** with conda, pipenv, pyenv etc. **Containerize** with Docker. Consider Kubeflow, Seldon Core etc to scale.
5. Consider using **cloud services** such as AWS Sagemaker, GCP – Vertex AI, Azure for development and deployment.
6. Monitor input data for **Data Drift**. Monitor target drift and model drift as well.
7. Monitor for **model performance degradation** post deployment. Capture model diagnostics specific to algorithm. Example: For linear regression, capture

coefficients, vif, r2 etc. For random forest classification, probability scores, capture rates with deciles etc.

8. Consider having a **staging environment** for critical end user facing projects, where model results are consumed live. Perform User Acceptance Testing (UAT) where needed.

Not every Data Science project is successful. There can be various non-technical reasons for that.

Try answering these questions and get it clarified ideally in the beginning of the project. This is benefit not just you, but your project team members and your company / client as well.

1. Who are the end users? How often will they use it?

python



Do they know about ML? Do they need to be educated.

2. How many people will be accessing the model results?

python



For larger audience, your ML deployment should be able to scale.

3. How long will it be in use and how many users should it support?

python



To be cost effective, enable adoption and usage for long term.

4. What parts can be modularized / templated, if you want to replicate for other products / regions?

python



You will have competitive advantage **if** able to reproduce **and** scale quick.

5. How much data will be generated?

python



```
1 | With time you might get newer data, you will need DB systems
2 | that can handle such volumes.
3 |
4 | For example: Data about user actions on a website can have large number of
5 | data points generated by the hour.
```

6. How often the model needs to be retrained after productionizing?

python



Budget this cost **as** well. They might **not** even know ML models need to be maintained / retrained. So, better **set** the expectations right.

7. Do you need a staging environment?

python



For customer facing models, ensure continuity. Always test before deployment. Also to replicate bugs.

8. Are your model's performance monitored?

python



Cross validation **is not** sufficient, monitor on real world results **as** well.

9. What other skillsets are needed to productize?

python



Do you need software engineers, data architects, ML ops / Dev Ops engineers, UI/UX developers? **is** RACI defined?

The [Machine Learning Plus Complete ML Mastery](https://edu.machinelearningplus.com/courses/machine-learning-plus-university-612383030cf2399aba22eaa3)

([https://edu.machinelearningplus.com/courses/machine-learning-plus-university-](https://edu.machinelearningplus.com/courses/machine-learning-plus-university-612383030cf2399aba22eaa3)

[612383030cf2399aba22eaa3](https://edu.machinelearningplus.com/courses/machine-learning-plus-university-612383030cf2399aba22eaa3)) Courses, features the ideal learning path if you want to succeed in a career in Data Science.

If you are struggling with understanding 'tough' ML and Stats concepts, at ML+ you will get the most clear, complete and straight forward explanations.

Demand for Data Scientists [far outweigh](https://hbr.org/2022/07/is-data-scientist-still-the-sexiest-job-of-the-21st-century) (<https://hbr.org/2022/07/is-data-scientist-still-the-sexiest-job-of-the-21st-century>) supply.

All you need to do is to take the first few steps and start a brand new career. [Start now!](https://edu.machinelearningplus.com/courses/machine-learning-plus-university-612383030cf2399aba22eaa3) (<https://edu.machinelearningplus.com/courses/machine-learning-plus-university-612383030cf2399aba22eaa3>).



WRITTEN BY

Selva Prabhakaran →

Related Articles